

Project Report Assignment 1

Sixtine Lyonnet

April 2025

Abstract

This report outlines the work carried out for the Raytracer Project. The methodology used was detailed in the textbook of the course CSC 3S006: Computer Graphics.

Contents

1	Introduction	2
2	Lab1	3
3	Lab2	4
4	Lab3	5
5	Lab4	6

1 Introduction

The code for this lab can be found in the file `main.cpp`. The classes in this document are the following:

- Vector, which was provided and defines a 3D vector and some operation on it.
- Ray, which corresponds to a ray of light with its origin and direction vector
- Object, which is a broad label regrouping both spheres and mesh objects.
- Bounding Box, providing a first acceleration structure. It contains the two extrema of our bounding box (B_{min} and B_{max}).
- Triangle Indices, which was provided
- Triangle Mesh (with a class BVH included in it). This class was also provided for a large part. We added the BVH class to it to further accelerate our code.
- Sphere, which contains sphere objects that can be seen on the scene.
- Scene, which contains an array of all the elements in a scene: the objects visible in it.

To run the code, you need to run the following lines on a terminal:

```
g++ -std=c++11 -o final_code final_code.cpp
./final_code
```

My work is based on the textbook provided to us by the professor during the course CSC 3S006: Computer Graphics. I also got some help from both the professor and the TA to help solve some bugs, as well as from Leal Koksai, who in particular gave me important hints on how to write two functions in my code. I specified where her help was given in comment in the code.

2 Lab1

For this first lab, we implemented the ray tracer with diffuse surfaces and reflection. As further work, we implemented transparent surfaces and Fresnel refraction.

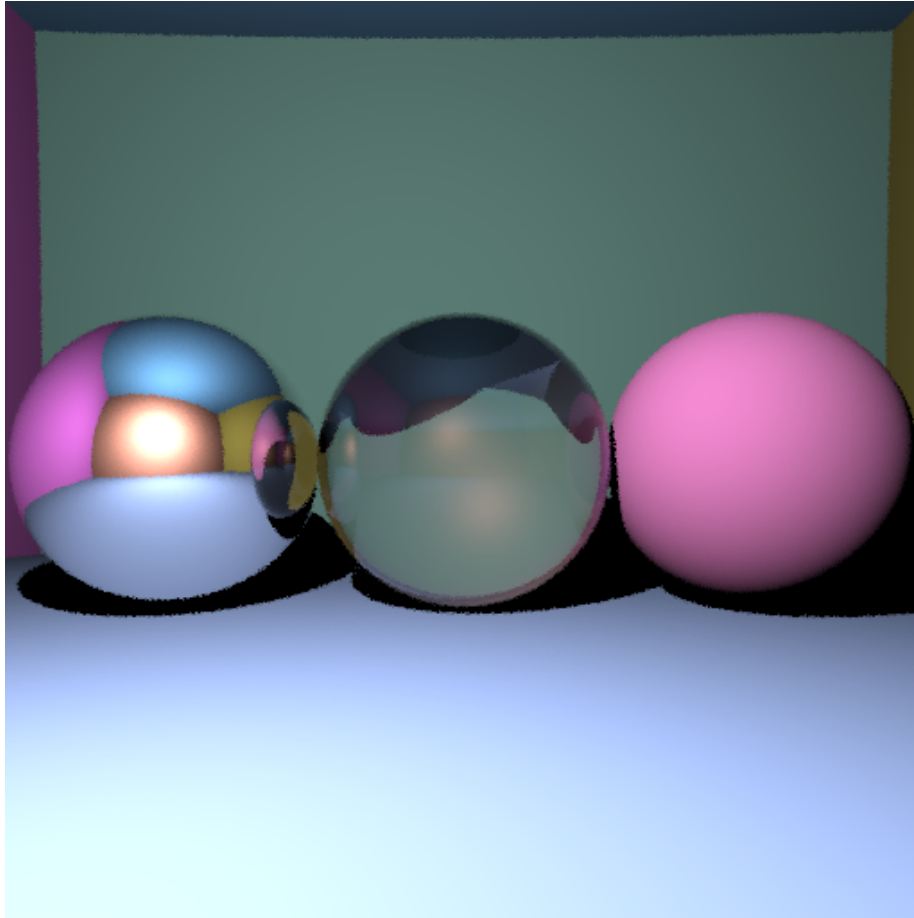


Figure 1: Image of spheres with Fresnel reflection, no indirect lighting, 4 rays per pixel, 512x512. Took 5 seconds.

3 Lab2

For this second lab, we implemented indirect lighting using the generation of random rays proportionally to the cosine factor around the normal of the surface and including this contribution to the image. We also implemented anti-aliasing this smooths the edges of the image.

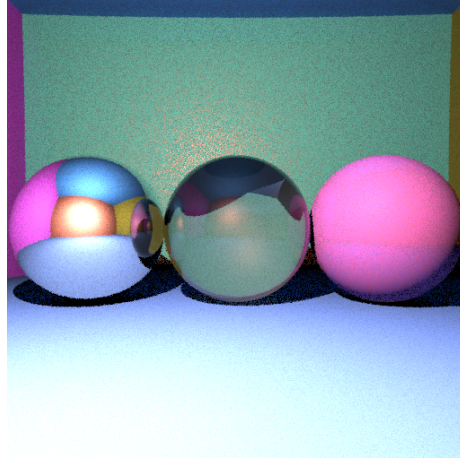


Figure 2: Image of spheres with Fresnel reflection, indirect lighting, 4 rays per pixel, 512x512. Took 14 seconds.

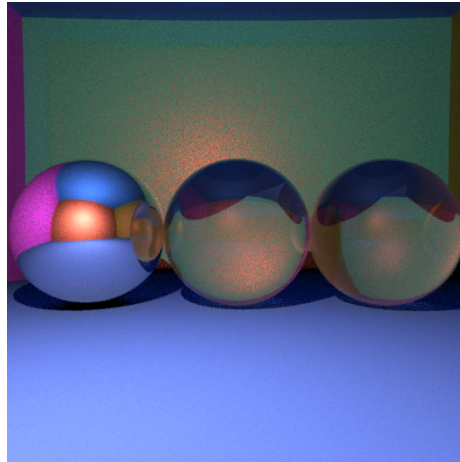


Figure 3: Image of spheres with Fresnel reflection, indirect lighting, 10 rays per pixel, 512x512. Took 160 seconds.

4 Lab3

For this third lab, we started implementing a support for meshes. We implemented the Möller-TRumbore ray-triangle-intersection algorithm to render meshes. We also implemented a bounding box to accelerate the rendering.

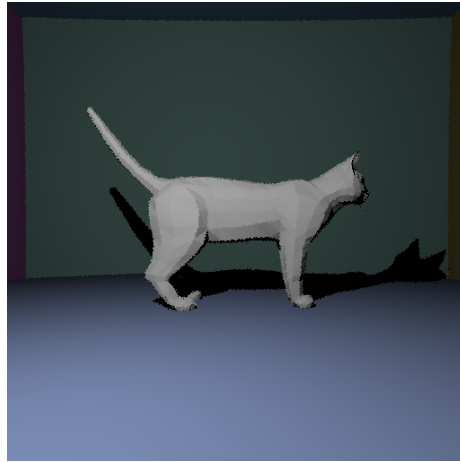


Figure 4: Cat image, 1 ray per pixel, 512x512 with shadow and indirect lighting, without bounding box. Took 754 seconds.

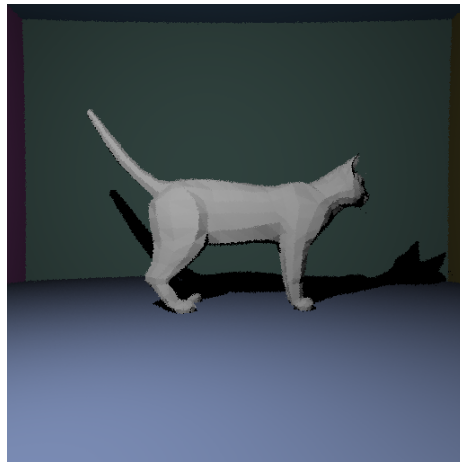


Figure 5: Cat image took, 1 ray per pixel, 512x512 with shadow and indirect lighting, with bounding box. Took 217 seconds.

5 Lab4

In the fourth lab, we implemented BVH to further accelerate the rendering.

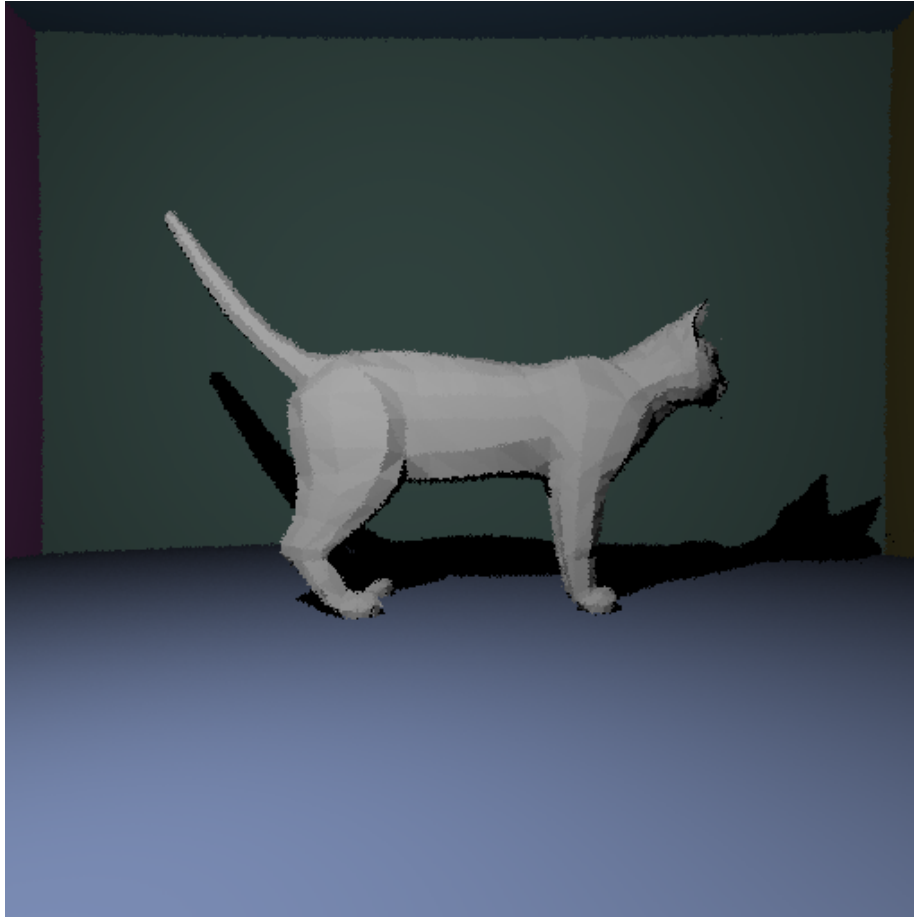


Figure 6: Cat image, 1 ray per pixel, 512x512 with shadow and indirect lighting, with Bounding Volume Hierarchies (BVH). Took 68 seconds.